

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_ace7351e-c4e\agent-afdc3bc0dcbcccf75.jsonl`
- SHA-256: `98492c5c00c43539c76eae7117327c0e841aa6d716458edb21065ed441380f61`
- Source modified: `2026-07-06T22:19:43+00:00`
- Imported at: `2026-07-08T16:00:07+00:00`
- project: `wf\_ace7351e-c4e`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T22:16:32.963Z)

Reviewing GitHub PR #52 "feat(policy): add F3 caption safety stage" (branch feat/f3-caption-safety -> main). ADR 0012 F3.
+299/-41, 4 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr52 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f3-caption-safety:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr52/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F3 DOES:

- policy.py: CaptionSafetyConfig + parse_caption_safety_config (from 'security_checks' key: unsafe_caption_terms[], unsafe_caption_patterns[]);
CaptionSafetyStage (Tier 3): fail-OPEN on invalid config (pass_ + warning + invalid_caption_safety_config=True); disabled if empty;
missing caption -> pass (caption_present=False); term match = casefold substring; pattern match = re.search(pattern, caption) (case-SENSITIVE
unless user adds (?i)); regexes validated via re.compile at parse time (invalid regex -> ValueError -> fail-open).
- telethon_client.py: _evaluate_topic_preferences RENAMED to _evaluate_policy_stages, now runs [TopicPreferenceStage(), CaptionSafetyStage()]
through evaluate_pipeline() and returns the list; live+backfill wiring: 'if policy_results: reasons.extend(...); matched = policy_results[-1].passed'.
- Reads security_checks from the RAW policy config (preserved by F1's save merge).
on_new_message/backfill_user are '# pragma: no cover'.

CONFIRMED by the main reviewer: gate green (523 passed @ 95.77%); ReDoS is REAL ? the valid pattern '(a)+\$' on a 40-char caption hangs
re.search past a 6s timeout (rc=124), blocking the synchronous event loop. Feature is dormant today (no command sets security_checks).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario, ranked.
Severity: high (real bug/regression/exploitable-now), medium (real but latent / host-wide-DoS-once-configurable / design), low (hygiene).

DIMENSION: ReDoS + regex/term-scanning security.

SEED (CONFIRMED, assess severity): unsafe_caption_patterns are user-supplied regexes evaluated via re.search on the SHARED asyncio event loop
with no timeout/complexity bound. A valid-but-pathological pattern (passes re.compile) + a crafted caption causes catastrophic backtracking that
hangs the loop. On a multi-user host (README: <100 users on one host, shared manager loop), one account's pattern stalls ALL users' ingestion.
Investigate: reachability (is there any shipped command/path that lets a user set security_checks today? or only save_policy_config/manual edit?);
does re.compile-at-parse catch ReDoS? (no ? ReDoS patterns are valid); caption length bound

(Telegram captions up to ~1024/4096 chars); is the loop truly blocked (re.search is sync)? whether backfill has the same exposure. Also: term matching casefold vs pattern case-sensitivity inconsistency; any injection/metadata-leak in StageResult (matched_caption_pattern echoes the user's own pattern - fine). Recommend concrete mitigation (timeout via regex module / executor+deadline, pattern length/count caps, complexity limits). Give a considered severity. Report findings.

2. assistant (2026-07-06T22:16:37.126Z)

I'll review PR #52's F3 caption safety stage, focusing on ReDoS and regex/term-scanning security. Let me start by reading the diff.

3. user (2026-07-06T22:16:39.559Z)

```
8e7acb6 feat(policy): add caption safety stage
6d3d7a6 feat(policy): add F2 topic preferences (#51)
fb2d2ea feat(policy): add F1 pipeline primitives (#50)
---DIFF---
src/tg_video_filter/backfill.py      | 10 +--
src/tg_video_filter/policy.py        | 139 ++++++-----
src/tg_video_filter/telethon_client.py | 32 +----
tests/test_policy.py                 | 159 ++++++-----
4 files changed, 299 insertions(+), 41 deletions(-)
```

4. assistant (2026-07-06T22:16:41.964Z)

Let me read the key files. I'll read the policy.py changes and the wiring in telethon_client.py and backfill.py.

5. user (2026-07-06T22:16:44.013Z)

```
diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
index 86db324..9e2664c 100644
--- a/src/tg_video_filter/policy.py
+++ b/src/tg_video_filter/policy.py
@@ -9,6 +9,7 @@ from __future__ import annotations

import json
import logging
+import re
from dataclasses import dataclass, field
from typing import Protocol

@@ -77,6 +78,18 @@ class TopicPreferences:
    return bool(self.include or self.exclude)

+@dataclass(frozen=True)
+class CaptionSafetyConfig:
+    """Parsed ADR 0012 caption safety config."""
+
+    unsafe_terms: tuple[str, ...] = ()
+    unsafe_patterns: tuple[str, ...] = ()
+
+    @property
+    def enabled(self) -> bool:
+        return bool(self.unsafe_terms or self.unsafe_patterns)
+
+def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
+    """Parse ``topics`` from a raw policy config object."""
+    raw_topics = config.get("topics")
```

```

@@ -84,8 +97,8 @@ def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
    return TopicPreferences()
    if not isinstance(raw_topics, dict):
        raise ValueError("topics must be a JSON object")
-   include = _string_tuple(raw_topics.get("include"))
-   exclude = _string_tuple(raw_topics.get("exclude"))
+   include = _string_tuple(raw_topics.get("include"), field_name="topics.include")
+   exclude = _string_tuple(raw_topics.get("exclude"), field_name="topics.exclude")
    if not include and not exclude:
        return TopicPreferences()
    mode = raw_topics.get("mode", "any")
@@ -98,6 +111,40 @@ def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
    )

```

```

+def parse_caption_safety_config(config: dict[str, object]) -> CaptionSafetyConfig:
+    """Parse caption safety settings from ``security_checks``.
+
+    Expected shape:
+    ``{"security_checks":{"unsafe_caption_terms":[],"unsafe_caption_patterns":[]}}``.
+    """
+    raw_security = config.get("security_checks")
+    if raw_security is None:
+        return CaptionSafetyConfig()
+    if not isinstance(raw_security, dict):
+        raise ValueError("security_checks must be a JSON object")
+
+    unsafe_terms = _string_tuple(
+        raw_security.get("unsafe_caption_terms"),
+        field_name="security_checks.unsafe_caption_terms",
+    )
+    unsafe_patterns = _string_tuple(
+        raw_security.get("unsafe_caption_patterns"),
+        field_name="security_checks.unsafe_caption_patterns",
+    )
+    for pattern in unsafe_patterns:
+        try:
+            re.compile(pattern)
+        except re.error as exc:
+            raise ValueError(
+                "security_checks.unsafe_caption_patterns contains invalid regex "
+                f"{pattern!r}: {exc}"
+            ) from exc
+    return CaptionSafetyConfig(
+        unsafe_terms=unsafe_terms,
+        unsafe_patterns=unsafe_patterns,
+    )
+
+def load_topic_preferences(policy: Policy) -> TopicPreferences:
+    """Load topic preferences from ``policy.config_json``.
+    decoded = json.loads(policy.config_json)
@@ -106,19 +153,27 @@ def load_topic_preferences(policy: Policy) -> TopicPreferences:
    return parse_topic_preferences(decoded)

```

```

-def _string_tuple(value: object) -> tuple[str, ...]:
+def load_caption_safety_config(policy: Policy) -> CaptionSafetyConfig:
+    """Load caption safety settings from ``policy.config_json``.
+    decoded = json.loads(policy.config_json)
+    if not isinstance(decoded, dict):
+        raise ValueError("policy config_json must decode to a JSON object")
+    return parse_caption_safety_config(decoded)
+
+

```

```

+def _string_tuple(value: object, *, field_name: str) -> tuple[str, ...]:
    if value is None:
        return ()
    if not isinstance(value, list):
        raise ValueError("topic include/exclude values must be arrays")
    topics: list[str] = []
    + raise ValueError(f"{field_name} must be an array")
+ values: list[str] = []
    for item in value:
        if not isinstance(item, str):
            raise ValueError("topic include/exclude arrays must contain strings")
        topic = item.strip()
        if topic:
            topics.append(topic)
    return tuple(topics)
+ raise ValueError(f"{field_name} must contain strings")
+ text = item.strip()
+ if text:
+     values.append(text)
+ return tuple(values)

class TopicPreferenceStage:
@@ -189,6 +244,66 @@ def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str,
...]:
    return tuple(topic for topic in topics if topic.casefold() in haystack)

+class CaptionSafetyStage:
+    """ADR 0012 Tier 3 captured-caption term and pattern scanning."""
+
+    stage_name = "caption-safety"
+
+    async def evaluate(self, ctx: PolicyContext) -> StageResult:
+        try:
+            config = load_caption_safety_config(ctx.policy)
+        except (json.JSONDecodeError, ValueError) as exc:
+            log.warning(
+                "caption safety disabled due to invalid config policy_id=%s",
+                ctx.policy.id,
+                exc_info=True,
+            )
+            return StageResult.pass_(
+                self.stage_name,
+                f"invalid caption safety config ignored: {exc}",
+                invalid_caption_safety_config=True,
+            )
+        if not config.enabled:
+            return StageResult.pass_(self.stage_name, "caption safety disabled")
+
+        caption = (ctx.content_item.text or "").strip()
+        if not caption:
+            return StageResult.pass_(
+                self.stage_name,
+                "no caption to scan",
+                caption_present=False,
+            )
+
+        haystack = caption.casefold()
+        for term in config.unsafe_terms:
+            if term.casefold() in haystack:
+                return StageResult.fail(
+                    self.stage_name,
+                    f"unsafe caption term matched: {term}",
+                    caption_present=True,

```

```

+             match_type="term",
+             matched_caption_term=term,
+         )
+
+     for pattern in config.unsafe_patterns:
+         if re.search(pattern, caption):
+             return StageResult.fail(
+                 self.stage_name,
+                 "unsafe caption pattern matched",
+                 caption_present=True,
+                 match_type="pattern",
+                 matched_caption_pattern=pattern,
+             )
+
+     return StageResult.pass_(
+         self.stage_name,
+         "caption passed safety checks",
+         caption_present=True,
+         checked_terms=len(config.unsafe_terms),
+         checked_patterns=len(config.unsafe_patterns),
+     )
+
+
+ async def evaluate_pipeline(
+     ctx: PolicyContext,
+     stages: list[PolicyStage],
+
@@ -204,12 +319,16 @@ async def evaluate_pipeline(

__all__ = [
+     "CaptionSafetyConfig",
+     "CaptionSafetyStage",
+     "PolicyContext",
+     "PolicyStage",
+     "StageResult",
+     "TopicPreferenceStage",
+     "TopicPreferences",
+     "evaluate_pipeline",
+     "load_caption_safety_config",
+     "load_topic_preferences",
+     "parse_caption_safety_config",
+     "parse_topic_preferences",
+ ]

```

6. user (2026-07-06T22:16:47.884Z)

```

diff --git a/src/tg_video_filter/backfill.py b/src/tg_video_filter/backfill.py
index 15a7aa2..64905bf 100644
--- a/src/tg_video_filter/backfill.py
+++ b/src/tg_video_filter/backfill.py
@@ -34,7 +34,7 @@ from tg_video_filter.filters import FilterEngine
 from tg_video_filter.models import Channel, DeliveryMode, User, VideoRecord
 from tg_video_filter.telethon_client import (
     _content_item_preview,
     _evaluate_topic_preferences,
-    _evaluate_policy_stages,
+    _extract_document_id,
     _message_text,
     _peer_id,
@@ -244,7 +244,7 @@ async def backfill_user( # pragma: no cover - requires live Telethon
 connection

     matched = result.matched
     reasons = list(result.reasons)
     if matched and source is not None:
-        topic_result = await _evaluate_topic_preferences(

```

```

+         policy_results = await _evaluate_policy_stages(
+             conn,
+             user.id,
+             source,
@@ -258,9 +258,9 @@ async def backfill_user( # pragma: no cover - requires live Telethon
connection
+             matched,
+         ),
+     )
-         if topic_result is not None:
-             reasons.append(topic_result.reason)
-             matched = topic_result.passed
+         if policy_results:
+             reasons.extend(result.reason for result in policy_results)
+             matched = policy_results[-1].passed

        record = VideoRecord(
            user_id=user.id,
diff --git a/src/tg_video_filter/telethon_client.py b/src/tg_video_filter/telethon_client.py
index 7b393f8..6dad7c4 100644
--- a/src/tg_video_filter/telethon_client.py
+++ b/src/tg_video_filter/telethon_client.py
@@ -44,7 +44,13 @@ from tg_video_filter.models import (
    User,
    VideoRecord,
)
-from tg_video_filter.policy import PolicyContext, StageResult, TopicPreferenceStage
+from tg_video_filter.policy import (
+    CaptionSafetyStage,
+    PolicyContext,
+    StageResult,
+    TopicPreferenceStage,
+    evaluate_pipeline,
+)

log = logging.getLogger(__name__)

@@ -272,7 +278,7 @@ def make_new_message_handler(
    matched = result.matched
    reasons = list(result.reasons)
    if matched and source is not None:
-        topic_result = await _evaluate_topic_preferences(
+        policy_results = await _evaluate_policy_stages(
            conn,
            user.id,
            source,
@@ -286,9 +292,9 @@ def make_new_message_handler(
        matched,
    ),
)
-    if topic_result is not None:
-        reasons.append(topic_result.reason)
-        matched = topic_result.passed
+    if policy_results:
+        reasons.extend(result.reason for result in policy_results)
+        matched = policy_results[-1].passed

    record = VideoRecord(
        user_id=user.id,
@@ -420,19 +426,21 @@ def _content_item_preview(
)

-async def _evaluate_topic_preferences(
+async def _evaluate_policy_stages(

```

```

    conn:aiosqlite.Connection,
    account_id:int,
    source:Source,
    content_item:ContentItem,
-) -> StageResult | None:
-    """Evaluate ADR 0012 topic preferences for a candidate item."""
+) -> list[StageResult]:
+    """Evaluate ADR 0012 policy stages for a candidate item."""
    policy = await db.get_default_policy(conn, account_id)
    if policy is None:
-        return None
-    return await TopicPreferenceStage().evaluate(
-        PolicyContext(content_item=content_item, source=source, policy=policy)
+    return []
+    _passed, results = await evaluate_pipeline(
+        PolicyContext(content_item=content_item, source=source, policy=policy),
+        [TopicPreferenceStage(), CaptionSafetyStage()],
+    )
+    return results

def _peer_id(peer: object) -> int:
@@ -468,6 +476,6 @@ __all__ = [
    "extract_video_meta",
    "make_new_message_handler",
    "_content_item_preview",
-    "_evaluate_topic_preferences",
+    "_evaluate_policy_stages",
    "_message_text",
]

```

7. assistant (2026-07-06T22:16:50.212Z)

Now let me look at `evaluate\_pipeline` and the surrounding policy code to understand the pipeline semantics (short-circuit behavior, how `passed` accumulates).

8. user (2026-07-06T22:16:51.150Z)

```

300         "caption passed safety checks",
301         caption_present=True,
302         checked_terms=len(config.unsafe_terms),
303         checked_patterns=len(config.unsafe_patterns),
304     )
305
306
307     async def evaluate_pipeline(
308         ctx: PolicyContext,
309         stages: list[PolicyStage],
310     ) -> tuple[bool, list[StageResult]]:
311         """Evaluate stages in order and short-circuit on the first failure."""
312         results: list[StageResult] = []
313         for stage in stages:
314             result = await stage.evaluate(ctx)
315             results.append(result)
316             if not result.passed:
317                 return False, results
318         return True, results
319
320
321     __all__ = [
322         "CaptionSafetyConfig",
323         "CaptionSafetyStage",
324         "PolicyContext",
325         "PolicyStage",
326         "StageResult",

```

```

327         "TopicPreferenceStage",
328         "TopicPreferences",
329         "evaluate_pipeline",
330         "load_caption_safety_config",
331         "load_topic_preferences",
332         "parse_caption_safety_config",
333         "parse_topic_preferences",
334     ]
335

```

9. assistant (2026-07-06T22:16:54.951Z)

`evaluate\_pipeline` short-circuits on first failure, so `results[-1]` is correct ? the last result is either the failing stage or the last passing stage. The wiring logic (`matched = policy\_results[-1].passed`) is sound.

Now the core question: the ReDoS seed. Let me look at what `content\_item.text` can hold (caption length) and confirm reachability of `security\_checks`. Let me check for any command that sets `security\_checks`.

10. user (2026-07-06T22:16:55.902Z)

```

C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\db.py:1317:   as ``topics`` /
``security_checks`` / ``deep_inspect`` instead of validating
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\policy.py:115:   """Parse caption safety
settings from ``security_checks``.
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\policy.py:118:
``{"security_checks":{"unsafe_caption_terms":[],"unsafe_caption_patterns":[]}}``.
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\policy.py:120:   raw_security =
config.get("security_checks")
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\policy.py:124:           raise
ValueError("security_checks must be a JSON object")
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\policy.py:127:
raw_security.get("unsafe_caption_terms"),
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\policy.py:128:
field_name="security_checks.unsafe_caption_terms",
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\policy.py:131:
raw_security.get("unsafe_caption_patterns"),
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\policy.py:132:
field_name="security_checks.unsafe_caption_patterns",
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\policy.py:139:
"security_checks.unsafe_caption_patterns contains invalid regex "

```

11. assistant (2026-07-06T22:16:58.310Z)

Let me look at db.py:1317 context (the save_policy_config merge that preserves security_checks) to confirm reachability.

12. user (2026-07-06T22:16:59.075Z)

```

1295 def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
1296     """Decode a policy config JSON object.
1297
1298     ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
1299     expected to be a JSON object so legacy flat ``FilterConfig`` keys and future
1300     nested stage config can coexist.
1301     """
1302     decoded = json.loads(raw)
1303     if not isinstance(decoded, dict):
1304         raise ValueError("policy config_json must decode to a JSON object")
1305     return decoded
1306
1307
1308 def _policy_config_dict_to_json(config: dict[str, object]) -> str:

```



```

1309         """Encode policy config deterministically and compactly."""
1310         return json.dumps(config, separators=(",", ":"), sort_keys=True)
1311
1312
1313     async def load_policy_config(conn: aiomysqlite.Connection, account_id: int) -> dict[str,
object]:
1314         """Load the raw default policy config as a JSON object.
1315
1316         Unlike :func:`load_filter_config`, this preserves future ADR 0012 keys such
1317         as ``topics`` / ``security_checks`` / ``deep_inspect`` instead of validating
1318         through the flat ``FilterConfig`` model.
1319         """
1320         policy = await get_default_policy(conn, account_id)
1321         if policy is None:
1322             return FilterConfig().model_dump(mode="json")
1323         return _policy_config_json_to_dict(policy.config_json)
1324
1325
1326     async def save_policy_config(
1327         conn: aiomysqlite.Connection, account_id: int, config: dict[str, object]
1328     ) -> None:
1329         """Persist the raw default policy config object."""
1330         existing = await get_default_policy(conn, account_id)
1331         await upsert_policy(
1332             conn,
1333             Policy(
1334                 id=existing.id if existing is not None else None,
1335                 account_id=account_id,
1336                 name=existing.name if existing is not None else "default",
1337                 config_json=_policy_config_dict_to_json(config),
1338                 is_default=True,
1339                 created_at=(existing.created_at if existing is not None else _now()),
1340                 updated_at=_now(),
1341             ),
1342         )
1343
1344
1345     # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
1346
1347
1348     async def upsert_destination(conn: aiomysqlite.Connection, dest: Destination) ->
Destination:
1349         """Insert or update a destination row by (account, provider, target)."""
1350         await conn.execute(
1351             """INSERT INTO destinations
1352             (account_id, provider, target, kind, title, is_output, created_at)
1353             VALUES (?, ?, ?, ?, ?, ?, ?)
1354             ON CONFLICT(account_id, provider, target) DO UPDATE SET

```

13. assistant (2026-07-06T22:17:02.241Z)

Now let me check who calls `save\_policy\_config` ? is there a bot command that lets a user set arbitrary config (and thus `security\_checks`)?

14. user (2026-07-06T22:17:03.789Z)

```

C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\db.py:1313:async def
load_policy_config(conn: aiomysqlite.Connection, account_id: int) -> dict[str, object]:
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\db.py:1326:async def save_policy_config(
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\db.py:1793:     await
save_policy_config(conn, user_id, raw_config)
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\db.py:1861:         "load_policy_config",
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\db.py:1862:         "save_policy_config",

```

15. user (2026-07-06T22:17:06.185Z)

```
1760         instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1761         Invalid ``config_json`` still raises from JSON/Pydantic validation; callers
1762         do not currently treat malformed persisted config as recoverable.
1763         """
1764         policy = await get_default_policy(conn, user_id)
1765         if policy is None:
1766             return FilterConfig()
1767         return FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))
1768
1769
1770     async def save_filter_config(conn: aioredis.Connection, user_id: int, cfg:
FilterConfig) -> None:
1771         """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
1772
1773         ADR 0012: future pipeline keys live in the same ``policies.config_json``
1774         object. Do not overwrite the whole blob with ``cfg.model_dump_json()``;
1775         merge the flat config fields into the existing object so unknown policy
1776         keys survive legacy ``/set`` command updates.
1777         """
1778         existing = await get_default_policy(conn, user_id)
1779         raw_config: dict[str, object] = {}
1780         if existing is not None:
1781             try:
1782                 raw_config = _policy_config_json_to_dict(existing.config_json)
1783             except (json.JSONDecodeError, ValueError):
1784                 log.warning(
1785                     "resetting malformed policy config while saving FilterConfig user=%s "
1786                     "policy_id=%s",
1787                     user_id,
1788                     existing.id,
1789                     exc_info=True,
1790                 )
1791             raw_config = {}
1792         raw_config.update(cfg.model_dump(mode="json"))
1793         await save_policy_config(conn, user_id, raw_config)
1794
1795
1796     # --- bulk helpers (used by the manager on startup) -----
1797
1798
1799     async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
```

16. assistant (2026-07-06T22:17:19.379Z)

So `save\_policy\_config` is only called via `save\_filter\_config` (the legacy `/set` shim), which merges flat FilterConfig keys ? it does not accept arbitrary `security\_checks`. This confirms the seed: there is **no shipped command** that sets `security\_checks` today. It can only be populated by manual DB edit / a future command. So the feature is dormant, which caps present-day severity. But `load\_policy\_config` preserves it if present, and the stage reads it. Let me confirm the ReDoS repro and check caption length bounds.

Let me confirm the content_item.text source (caption length). Let me check `\_message\_text` / ContentItem.

17. user (2026-07-06T22:17:20.730Z)

```
303:         text=text,
355:         link_text=link_text,
380: def _message_text(message: object) -> str | None:
381:     """Return caption/message text normalized for storage, or None if empty."""
408:     text=text,
```

18. user (2026-07-06T22:17:23.351Z)

```
380     def _message_text(message: object) -> str | None:
381         """Return caption/message text normalized for storage, or None if empty."""
382         text = getattr(message, "message", None)
383         if text is None:
384             text = getattr(message, "raw_text", None)
385         if not isinstance(text, str):
386             return None
387         text = text.strip()
388         return text or None
389
390
391     def _content_item_preview(
392         account_id: int,
393         source: Source,
394         message_id: int,
395         document_id: int | None,
396         meta: VideoMeta,
397         text: str | None,
398         matched: bool,
399     ) -> ContentItem:
400         """Build the ContentItem shape used by policy stages before insert."""
401         assert source.id is not None
402         return ContentItem(
403             account_id=account_id,
404             source_id=source.id,
405             provider=source.provider,
406             provider_message_id=str(message_id),
407             content_type="video",
408             text=text,
409             document_id=str(document_id) if document_id is not None else None,
```

19. assistant (2026-07-06T22:17:26.719Z)

The caption comes directly from the Telegram message (`message.message` / `raw\_text`), attacker-controlled by whoever posts in the source channel, up to Telegram's caption limit (~1024, or 4096 for premium). No length bound applied before regex. Now let me run the ReDoS repro to confirm the loop-blocking behavior.

20. user (2026-07-06T22:17:35.878Z)

written

21. assistant (2026-07-06T22:17:38.024Z)

Let me check the Policy model fields to make sure my construction is valid.

22. user (2026-07-06T22:17:39.667Z)

```
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\policy.py:22:class PolicyContext:
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\models.py:123:class Source(BaseModel):
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\models.py:149:class ContentItem(BaseModel):
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\models.py:186:class Policy(BaseModel):
```

23. assistant (2026-07-06T22:17:43.083Z)

My FakeItem/FakeSource duck-typing works since PolicyContext is a plain dataclass. Let me run the repro. `time.sleep` is blocked in the foreground per env, but this is a Python script (asyncio.sleep inside), so it's fine.

24. user (2026-07-06T22:18:17.133Z)

rc=124

25. assistant (2026-07-06T22:18:22.044Z)

Confirmed ? the process is killed by the 30s `timeout` (rc=124). `re.search("(a+)+\$", "a"\*40 + "!"` catastrophically backtracks. Because `evaluate` is `async` but the `re.search` call is ****synchronous CPU work with no `await` inside it****, the asyncio event loop is fully blocked for the duration ? the concurrent "other" task never got to tick. This is a genuine host-wide event-loop stall.

Let me confirm the loop-blocking claim more precisely with a shorter, bounded caption that finishes in a measurable (but large) time, to show the "other task starved" effect quantitatively rather than just timing out.

26. user (2026-07-06T22:18:51.890Z)

caption len=29 regex ran 19.95s; other-task ticks DURING regex = 0 (expected ~997 if loop free)
rc=0

27. assistant (2026-07-06T22:18:58.668Z)

Definitive proof: a 29-char caption caused `re.search` to run for ****19.95 seconds****, during which the concurrent async task got ****0 ticks**** (expected ~997 if the loop were free). The event loop is fully starved. On the shared manager loop (README: <100 users, one loop), one account's pattern + one crafted caption in any watched channel stalls ingestion for all users.

Now let me verify the two secondary points: (1) the casefold/case-sensitivity inconsistency, and (2) confirm `matched\_caption\_pattern` echoes only the user's own pattern (not attacker input). Let me also check whether there's any pattern-count or length bound anywhere. There isn't ? confirmed from the diff. Let me quickly verify the case-sensitivity behavior with a repro.

28. user (2026-07-06T22:19:06.341Z)

TERM 'BadWord' vs term 'badword': (False, 'unsafe caption term matched: badword')
PAT 'BADWORD' vs pattern 'badword': (True, 'caption passed safety checks')
rc=0

29. assistant (2026-07-06T22:19:17.766Z)

Confirmed the inconsistency: a `badword` ****term**** blocks the "BADWORD" caption (case-insensitive casefold), but a `badword` ****pattern**** does NOT (case-sensitive `re.search`, passes). For a ***safety*** stage, this is a real evasion gap ? an operator configuring a pattern to block something gets silently case-sensitive matching, and an attacker changes case to bypass. It's a security-relevant footgun but latent (dormant feature, and operator-configurable), so medium.

Now let me finalize severity on the ReDoS. Reachability: no shipped command sets `security\_checks`; only reachable via manual DB edit or a future command. The feature is dormant today, so it is ****not exploitable-now**** in the "shipped default path" sense. Per the ground rules, "high" = exploitable-now; "medium" = host-wide-DoS-once-configurable. This is precisely the medium definition: real, host-wide DoS, but gated behind config that no current command exposes. I'll rank ReDoS as the top finding at ****medium**** (borderline high ? it's a self-inflicted-config caveat, but the ***trigger*** is an attacker-controlled caption from any watched channel, not the operator's own input, which is what makes it dangerous once enabled).

Let me confirm the backfill path has identical exposure (it calls the same `\_evaluate\_policy\_stages`). From the diff: yes, `backfill.py:247` calls `\_evaluate\_policy\_stages` ? same `CaptionSafetyStage` ? same unbounded `re.search`. Confirmed both paths.

One more: verify `re.compile` at parse time does NOT catch ReDoS (patterns are valid) ? already proven since `(a+)+\$` compiled fine and still hung. Let me write up findings.

30. user (2026-07-06T22:19:43.145Z)

Structured output provided successfully